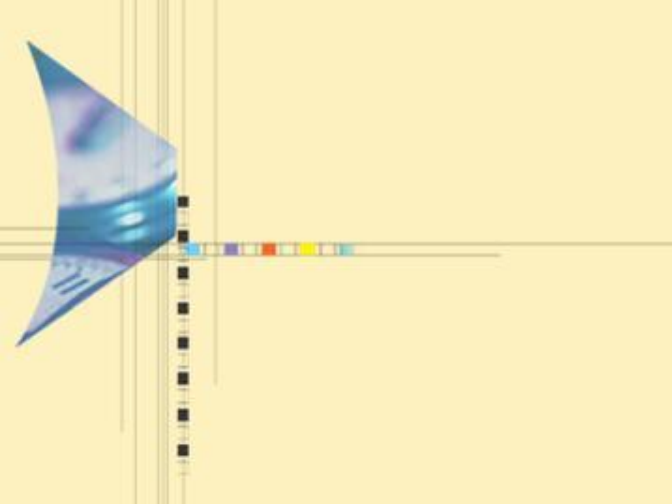




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法时间估算方法

1.4 求解递推关系



主要内容



1.3 算法运行时间的估算方法

- 算法运行时间估算的一般策略
- 平摊分析方法
- 输入大小与问题实例



1.3.1 算法运行时间估算的一般策略

总的目标：统计基本操作执行总次数。

(1) 计算迭代次数

- 诸如搜索、排序、矩阵乘法等算法，算法的运行时间通常和 while 及类似循环结构的执行次数成正比，此时，计算迭代次数能很好表明算法的运行时间。
- 具体计算迭代次数时，可以通过找到算法中执行最多的一个或多个语句来实现，估计它们的执行次数即可。

例：算法1.11 COUNT4

算法1.11 COUNT4

输入: $n=2^k$, k 为某个正整数.

输出: 第4步的执行次数count.

```
1. count  $\leftarrow$  0
2. while  $n \geq 1$ 
3.     for  $j \leftarrow 1$  to  $n$ 
4.         count  $\leftarrow$  count + 1
5.     end for
6.      $n \leftarrow n/2$ 
7. end while
8. return count
```



例：算法1.11 COUNT4

- 算法COUNT4 包含两个嵌套的循环，外层为while循环，内层为for 循环。包含一个变量count, count记录算法当某个k, $n=2^k$, 执行的总的迭代次数.
- 外层 **while** 循环总共执行 $k+1$ 次, $k=\log n$. 内层 **for** 循环依次执行 $n, n/2, n/4, \dots, 1$ 次。因此，第4步执行总次数为：

$$\sum_{j=0}^k \frac{n}{2^j} = n \sum_{j=0}^k \frac{1}{2^j} = n(2 - \frac{1}{2^k}) = 2n - 1 = \Theta(n)$$

- 由于算法运行时间和迭代次数成正比，因此为 $\Theta(n)$.

1.3.1 算法运行时间估算的一般策略

(2) 计算基本运算的频度

- 某些算法，通过循环迭代次数估算算法时间比较麻烦甚至不可能，此时，就是通过统计基本运算的频度的方式来估算。
- 这类方法一般分两步：首先确定基本运算，然后应用渐近表达式来确定该运算的阶，这个阶也就是算法运行时间的阶。

例如：

- MERGE算法中的元素中元素赋值运算，就代表了算法的运行时间。虽然在一般的MERGE算法中，元素比较运算一般不是基本运算，但是当这个算法用于合并大小相同的两个数组时（大小分别大约为 $n/2$ ），再次特例下，元素比较运算页是基本运算。
- 将MERGE用于BOTTOMUP SORT算法，就是MERGE上述情况。

例：对BOTTOMUPSORT算法运行时间的分析

- 首先，BOTTOMUPSORT算法中每次循环都调用MERGE算法，因此的基本操作继承自MERGE算法。
- 其次，针对BOTTOMUPSORT算法特例（要合并的两个子数组大小大体相等），比较运算在相差一个常数因子的意义下具有最大频度，因此有把握选择元素比较运算作为基本运算。
- 根据观察1.5的结论，当 n 是2的幂时，算法的元素比较次数在 $(n \log n)/2$ 和 $n \log n - n + 1$ 之间。
- 因此，当 n 是2的幂时，元素总比较次数的阶为 $\Omega(n \log n)$ 和 $O(n \log n)$ ，也就是 $\Theta(n \log n)$ 。
- 可以证明，当 n 不是2的幂时，上述结论同样成立。
- 因此，算法的运行时间为 $\Theta(n \log n)$ 。

1.3.1 算法运行时间估算的一般策略

(3) 使用递推关系

- 在递归算法中，常常使用递推关系的形式给出界定运行时间的函数，即一个函数的定义包含了函数本身。
- 例如，在BINARYSEARCH算法中，如果令 $C(n)$ 为一个大小为 n 的实例中要执行的比较运算的次数，则算法中比较运算的次数可用如下递推式：

$$C(n) \leq \begin{cases} 1 & \text{if } n = 1 \\ C(\lfloor n/2 \rfloor) + 1 & \text{if } n \geq 2 \end{cases}$$

求解递推式

$$\begin{aligned} C(n) &\leq C(\lfloor n/2 \rfloor) + 1 \\ &= C(\lfloor \lfloor n/2 \rfloor / 2 \rfloor) + 1 + 1 \\ &= C(\lfloor n/4 \rfloor) + 1 + 1 \\ &\quad \dots\dots \\ &= \lfloor \log n \rfloor + 1 \end{aligned}$$

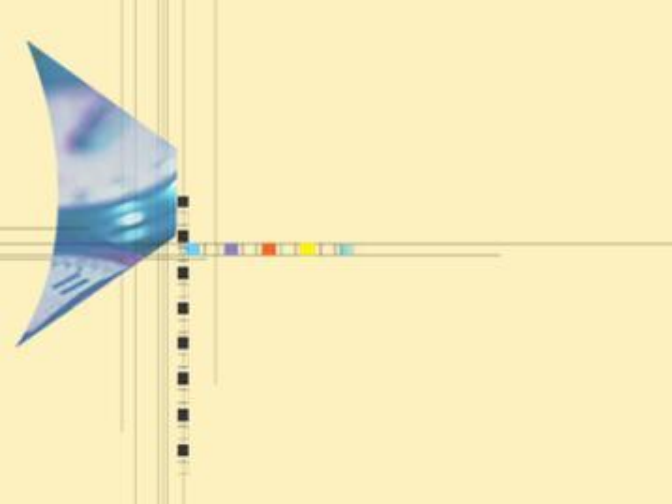
也就是 $C(n) \leq \lfloor \log n \rfloor + 1$. 因此， $C(n) = O(\log n)$. 由于元素比较运算是算法中的基本运算，因此算法的时间复杂性为 $O(\log n)$.

1.3.2 平摊分析 (Amortized analysis)

- 在许多算法中，无法用 Θ -符号表达时间复杂性以得到一个运行时间的精确界。此种情况，一般会用 O -符号来估计运行时间的上界。但是，有些时候，这个上界高估了，即使考虑最坏情况，算法可能也比估计的要快得多。
- 也就是，在分析算法最坏情形时间复杂度的时候，通常用关键操作的最坏情形执行时间，乘上这个操作的调用次数，来得到这个算法的运行时间上界。但是这样的分析并不总是奏效，往往高估了运行时间上界。
 - 考虑一个这样的算法，算法中的一种运算反复执行时具有如下特性：它的运行时间始终在波动。如果这一运算在大多数时候运行得很快，只是偶尔要花大量时间。又假定找到该算法的精确界虽然可能，但是非常困难，那么就预示着要使用平摊分析方法了。
- 平摊分析，能够帮助我们更准确的分析一个算法的运行时间上界。

什么是平摊分析?

- 在平摊分析中, 执行一系列操作所需要的时间是通过执行的所有操作求平均而得出的。
- 平摊分析可以用来证明在一系列操作中, 通过对所有操作求平均之后, 即使其中单一的操作具有较大的代价, 平均代价还是很小的。
- 平摊分析不牵涉到概率, 平摊分析保证在最坏情况下, 每个操作具有的平均性能。
- 常见的平摊分析方法: 聚集分析、记账法、势能法。



平摊分析方法 (1) —— 聚集分析



栈S上的三种操作

- **PUSH(S, x):** 将 x 压入 S
 - 运行时间 **$O(1)$**
- **POP(S):** 弹出栈顶
 - 运行时间 **$O(1)$**
- **MULTIPOP(S, k):** 弹出栈顶 k 个对象

MULTIPOP(S, k)

1 while not STACK-EMPTY(S) and $k \neq 0$

2 do POP(S)

3 $k \rightarrow k - 1$

- 运行时间 **$O(\min(k, s))$** , s 为栈中对象个数



n 个栈操作的平摊时间

- 设有 n 个栈操作 (**PUSH**、**POP**、**MULTIPOP**) 的序列，作用于初始为空的栈 S 。
- 总的运行时间的界是什么？
 - 每个操作都可能是**MULTIPOP**
 - 每个**MULTIPOP** 的运行时间是 $O(\min(k, s)) = O(n)$
 - 总的运行时间的上界为 $O(n^2)$
 - 这是一个紧的上界吗？



n 个栈操作的平摊时间

- 只有**PUSH**操作增加栈**S**中的对象个数
- 所有**POP**和**MULTIPOP**弹出的对象数不会弹出多于**PUSH**入栈的对象数
- 故总的运行时间为 $O(n)$
 - 所有**PUSH**入栈的对象数为 $O(n)$
 - 所有**POP**和**MULTIPOP**弹出的对象数也为 $O(n)$
- 每个**PUSH**、**POP**和**MULTIPOP**操作的平摊时间： $O(n)/n = O(1)$
- 聚集法：通过总运行时间求平均得到平摊时间，不需要对操作序列的概率分布做假设。

二进制计数器

- 计数器 $A[0 \dots k-1]$ 表示为 k 位二进制位的数组

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

- 操作 INCREMENT 实现计数器加一

- 运行时间 $O(k)$

- n 个 INCREMENT 操作序列的运行时间
 $O(nk)$ (紧吗?)

INCREMENT(A)

1 $i \leftarrow 0$

2 while $i < \text{length}[A]$ and $A[i] = 1$

3 $A[i] \leftarrow 0$

4 $i \leftarrow i + 1$

5 end while

6 if $i < \text{length}[A]$

7 then $A[i] \leftarrow 1$

8 end if

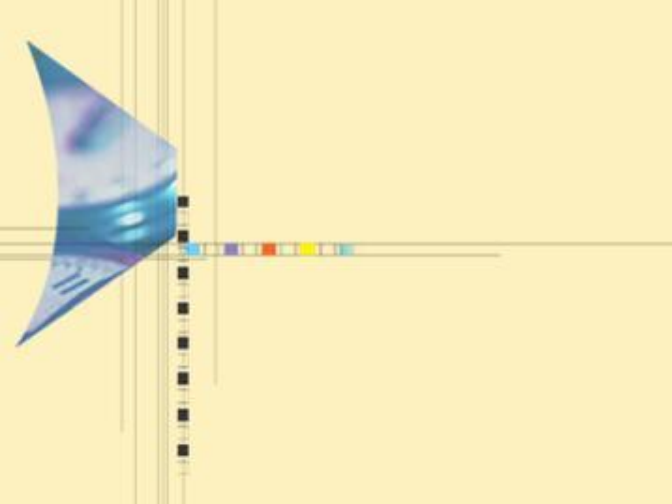
INCREMENT操作的平摊时间

- n 个INCREMENT操作序列的运行时间应为： $O(n)$
- 观察INCREMENT 操作序列
 - 每1次操作， $A[0]$ 都反转；
 - 每2次操作， $A[1]$ 反转；
 - 每 2^i 次操作， $A[i]$ 反转；
- 于是，总反转次数为

$$\sum_{i=0}^{\lfloor \log(n) \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

- 总时间 $O(n)$
- 每个操作平摊时间 $O(n) / n = O(1)$

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31



平摊分析方法 (2) ——记账法



记账法

- 对不同的操作赋予不同的费用，某些操作的费用比它们的实际代价或多或少。
- 我们对一个操作的收费的数量称为平摊代价。
 - 当一个操作的平摊代价超过了它的实际代价时，两者的差值就被当作存款，并赋予数据结构中的一些特定对象，可以用来补偿那些平摊代价低于其实际代价的操作。
 - 记账法与聚集分析的区别：
聚集分析中的所有操作都具有相同的平摊代价。
- 数据结构中存储的总存款等于总的平摊代价同总的实际代价之差。注意：总存款不能是负的。（因为我们期望总的平摊代价是总的实际代价的上界。）



n 个栈操作的平摊时间

操作	实际代价	平摊代价
PUSH	1	2
POP	1	0
MULTIPOP	$\min(k,s)$	0

- 对**PUSH**操作多收费，多出来的1元钱放在入栈的对象上，待**POP**和**MULTIPOP**把该对象弹出栈时，恰好每个对象上的1元前抵消了操作的实际代价
- 因为栈**S**中对象数不可能为负，即存款不会小于0
 - 保证 **n** 个操作平摊时间之和是 **n** 个操作实际时间的上界
- **n** 次操作总的平摊代价不超过 **$2n$** ，所以总时间 **$O(n)$** .

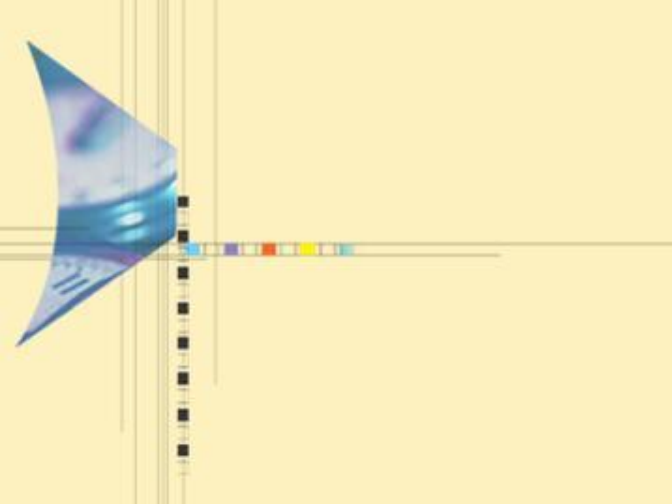
二进制计数器上的记账法分析

- 两个问题
 - 如何对**INCREMENT**收费? (平摊代价)
 - 收的费用放在哪里? (如何平摊)



二进制计数器上的记账法分析

- 注意到，每次**INCREMENT**只会把一个**0**反转为**1**，但可能把多个**1**反转为**0**。
 - **INCREMENT**的实际代价为反转的次数
 - **INCREMENT**平摊代价为**2**，用于把**0**反转为**1**，并把剩余的**1**元前存放在反转成**1**的位上。
 - 把**1**反转成**0**的实际代价由存放在**1**上的**1**元钱支付
- $A[1..k-1]$ 数组中，**1**的个数（=存款）不可能为负，总存款不可能为负数，因而平摊代价之和仍然是实际代价之和的上界。
- n 次操作最多将 n 个**0**反转为**1**，因此总平摊代价不超过 $2n$ ，因此总时间 $O(n)$ 。



平摊分析方法 (3) ——势能法



势能法 (potential method)

- 势能法和记账法，既有联系又有差别。势能法也可以理解为对某些操作多做收费。但与记账法不同的是，势能法多收的费用体现为势能的增加，而不是把存款存储在某个特定的对象之上，这种势能的增加是体现在整个数据结构之上的，当后续的操作使得势能减少时，释放出来的势能可用于抵消该操作的额外开销。
 - 不是将已预付的费用作为存在数据结构特定对象中存款来表示，而是表示成一种“势能”或“势”，它在需要时可以释放出来，以支付后面操作的额外开销。
 - 势是与整个数据结构而不是其中的个别对象发生联系的。（区别于记账法）

势函数 Φ

- 对一个初始数据结构 D_0 执行 n 个操作。对每个 i ，设 c_i 为每个操作的实际代价， D_i 为对数据结构 D_{i-1} 执行第 i 个操作的结果。
- 势函数 Φ 将每个数据结构 D_i 映射为一个实数 $\Phi(D_i)$ ，即与 D_i 相联系的势。
- 第 i 个操作的平摊代价 a_i 根据势函数 Φ 定义为

$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

- 即每个操作的平摊代价为其实际代价 c_i 加上由于该操作所增加的势。 n 个操作的总的平摊代价为

$$\sum a_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

势函数 Φ

- 如果势函数 Φ 使得对所有 n , 有 $\Phi(D_n) \geq \Phi(D_0)$, 则总平摊代价 $\sum a_i$ 就是总实际代价 $\sum c_i$ 的一个上界
 - 通常为了方便起见会定义 $\Phi(D_0) = 0$ (不是必须的)
 - 正的势差存储势
 - 如果第 i 个操作的势差 $\Phi(D_i) - \Phi(D_{i-1})$ 是正的, 则平摊代价 a_i 表示对第 i 个操作多收了费, 同时数据结构的势也随之增加了。
 - 负的势差不足收费
 - 如果势差是负值, 则平摊代价就表示对第 i 个操作的补足收费, 这是通过减少势来支付该操作的实际代价。
 - 平摊代价依赖于所选择的势函数 Φ 。
 - 不同的势函数可能会产生不同的平摊代价, 但它们都是实际代价的上界。最佳势函数的选择取决于所需的时间界。

栈操作——势能法分析

- 定义势能函数为栈中对象的个数
 - 开始时要处理是空栈 D_0 ，所以 $\Phi(D_0) = 0$
 - 栈中的对象数始终非负，所以 $\Phi(D_i) \geq 0 = \Phi(D_0)$
 - 以 Φ 表示的 n 个操作的平摊代价的总和，那么 Φ 就是总的实际代价的一个上界
- 对于作用于一个包含 s 个对象的栈上的第 i 个操作
- 如果PUSH，则
 - 势差为 $\Phi(D_i) - \Phi(D_{i-1}) = (s+1) - s = 1$
 - 平摊代价为 $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1+1=2$

栈操作——势能法分析

- 如果是MULTIPOP(S, k)

- 该操作的实际代价为 $c_i = \min(s, k)$
- 势差为 $\Phi(D_i) - \Phi(D_{i-1}) = -\min(s, k)$
- 平摊代价为

$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = \min(s, k) - \min(s, k) = 0$$

- 类似的，如果是POP，平摊代价也为0
- 三种栈操作每种和平摊代价都是 $O(1)$ ，这样包含 n 个操作的序列的总平摊代价就是 $O(n)$ 。
- 已经证明 n 个操作的平摊代价的总和是总的实际代价的一个上界。所以 n 个操作的最坏情况代价为 $O(n)$ 。

二进制计数器——势能法分析

- 计数器 $A[0 \dots k-1]$ 表示为 k 位二进制位的数组

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

- 操作 INCREMENT 实现计数器加一

- 如何定义势?

INCREMENT(A)

1 $i \leftarrow 0$

2 while $i < \text{length}[A]$ and $A[i] = 1$

3 $A[i] \leftarrow 0$

4 $i \leftarrow i + 1$

5 end while

6 if $i < \text{length}[A]$

7 then $A[i] \leftarrow 1$

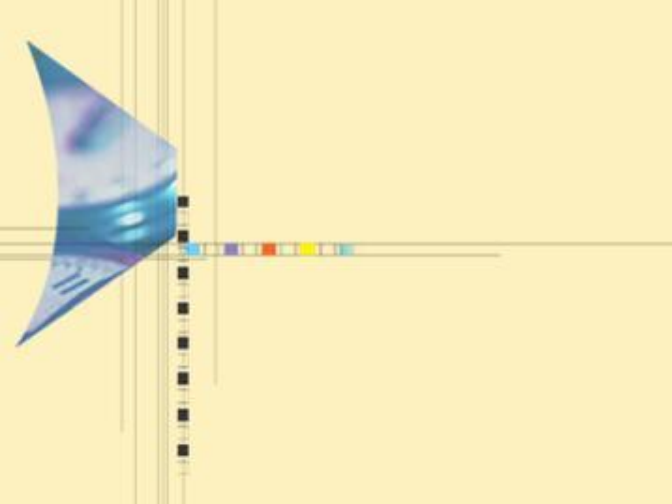
8 end if

二进制计数器—势能法分析

- 定义势函数为数组 $A[0...k-1]$ 中1的个数
计算 **INCREMENT** 的平摊代价
 - 设第 i 次操作把 t_i 个1反转为0，则实际代价为 $t_i + 1$
 - 设第 i 次操作后数组中1的个数为 b_i
 - 如果 $b_i = 0$ ，则第 i 次操作反转了 k 个1，有 $b_{i-1} = t_i = k$;
 - 如果 $b_i > 0$ ，则 $b_i = b_{i-1} - t_i + 1$ 。
 - 两种情形下都有: $b_i \leq b_{i-1} - t_i + 1$
- 势差为 $\Phi(D_i) - \Phi(D_{i-1}) \leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$
- 平摊开销为 $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \leq (t_i + 1) + (1 - t_i) = 2$

二进制计数器—势能法分析

- 设计数器中初始有 b_0 个1, n 次 INCREMENT 操作后有 b_n 个1。 ($0 \leq b_0, b_n \leq k$, k 为计数器位数)。因此,
 $\Phi(D_n) = b_n$, $\Phi(D_0) = b_0$
- 由于对所有的 i , $a_i \leq 2$
则有, n 次 INCREMENT 操作实际总代价
 $\sum c_i = \sum a_i - \Phi(D_n) + \Phi(D_0) \leq \sum 2 - b_n + b_0 = 2n - b_n + b_0$
- 只要 $k \leq O(n)$, 则总代价就为 $O(n)$
- 这里并没有要求 $\Phi(D_n) \geq \Phi(D_0)$, 也没有要求 $\Phi(D_0) = 0$



动态表及其上的平摊分析



动态表

- 在有些应用中，在开始的时候无法预知在表中要存储多少个对象。这就希望能根据对象的多少调整所需要的存储空间（多退少补）
- 表的动态扩张和收缩——动态表
 - 用平摊分析证明插入、删除操作的平摊代价是 $O(1)$
 - 动态表的具体结构可以是堆、栈、散列表等等
 - 假设我们用数组来存储动态表（不是链表）
 - 通过插入扩张和删除收缩
 - 保证未使用空间总是不多于整个分配空间的一定比例
- 先看只有插入操作的情形，再拓展也有删除操作

表扩张

算法 TABLE-INSERT

输入: $A[1..n]$, 存放待插入的 n 个元素

输出: T , 元素列表

1. $\text{num} \leftarrow 0$
2. **for** $i \leftarrow 1$ **to** n
3. **if** $\text{size}(T) = 0$ **then**
4. allocate T with 1
5. $\text{size}(T) \leftarrow 1$
6. **end if**
6. **if** $\text{num} = \text{size}(T)$ **then**
7. allocate new- T with $2 * \text{size}(T)$
8. insert all items in T into new- T
9. free(T)
10. $T \leftarrow \text{new-}T$
11. $\text{size}(T) \leftarrow 2 * \text{size}(T)$
12. **end if**
12. insert ($A[i]$, T)
13. $\text{num} \leftarrow \text{num} + 1$
14. **end for**

一次插入操作的代价

- 简单分析，一次插入操作的代价为 $O(n)$
 - 于是 n 次插入操作总代价为 $O(n^2)$
- 聚集分析，第 i 次插入操作的代价为
 - 当 $i-1$ 是 2 的幂时: $c_i = i = 2^k + 1. (k \leq \lfloor \log n \rfloor)$
 - 其他时候: $c_i = 1$
 - 总代价 $\sum_{i=1}^n c_i \leq n + \sum_{k=0}^{\lfloor \log n \rfloor} 2^k \leq n + 2n = 3n$
 - 平摊代价为 $3n/n = 3$
- 记账法，插入收费 3，其中剩余的 2 分别赋给表中最后插入的对象和一个已经没有存款的对象（刚好每个对象有存款 1）。
 - 当需要扩张表时，每个对象被复制一次，恰好用完存款。

表扩张时插入操作的平摊代价

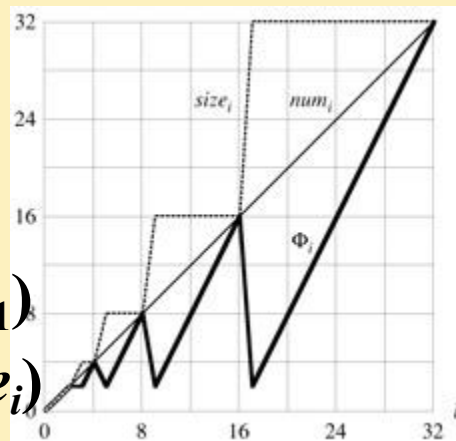
- 势函数: $\Phi(T) = 2num[T] - size[T]$

- 当第*i*个操作不触发表扩张, 则

$$\begin{aligned}a_i &= c_i + \Phi_i - \Phi_{i-1} \\&= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\&= 1 + (2 \cdot num_i - size_i) - (2(num_i - 1) - size_i) \\&= 3\end{aligned}$$

- 当第*i*个操作触发表扩张, 则

$$\begin{aligned}a_i &= c_i + \Phi_i - \Phi_{i-1} \\&= num_i + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\&= num_i + (2 \cdot num_i - 2(num_i - 1)) - (2(num_i - 1) - (num_i - 1)) \\&= num_i + 2 - (num_i - 1) = 3\end{aligned}$$



表扩张和收缩时的平摊代价

- 插入/删除操作：满时扩张、小于1/4时收缩
- 势函数
 - $\Phi(T) = \max(2num[T] - size[T], size[T]/2 - num[T])$
- 插入操作的平摊代价都小于3
 - 根据是否引起扩张来分别计算
- 删除操作的平摊代价都小于2
 - 根据是否引起收缩来分别计算
- 总平摊代价为 $O(n)$

插入/删除操作：满时扩张、小于1/3时收缩，如何定义势函数？

1.3.4 输入大小与问题实例

- 一个算法执行性能的测度通常是它的输入大小、顺序和分布等函数。其中最重要的就是输入大小。
- 输入大小的概念属于算法分析的实践部分，并且对它的解释已经约定成俗。
- 当讨论一个算法的问题时，通常讨论的这个问题的实例。因此，一个问题的实例就转变为求解该问题算法的输入。例如，将含有 n 个元素的数组 A 称为关于排序问题的一个实例。在讨论一个具体算法时，这个数组看作是算法的输入。

1.3.4 输入大小与问题实例

- 输入大小并不是输入的精确度量，一些常用的输入大小的测度约定如下：
 - 排序或搜索算法中，数组或者表中元素的数目；
 - 图算法中，图的边或顶点的数目，或者二者皆有；
 - 计算几何问题的算法中，点、边、线段、多边形等的数目；
 - 矩阵运算中，输入矩阵的维数；
 - 数论或者密码学算法中，表示输入数的比特数等。



1.3.4 输入大小与问题实例

- “不单一”的测度方法在比较两个算法所需的时间和空间时会带来一些不一致性。
- 例如，将两个 $n \times n$ 的矩阵相加的算法要执行 n^2 次两个数的加法运算，执行时间看上去是平方关系，但实际上是与其输入大小成线性关系。
- 再如，下来FIRST 和 SECOND算法，都循环 n 次，但是两个算法的输入不同，FIRST算法输入的是数组，时间复杂性是 $\Theta(n)$ 。SECOND算法是数论的算法，输入的是一个数，所以它的时间关于输入大小的指数时间复杂度算法。

再例

算法 1.14 FIRST

输入: 一个正整数 n 和一个数组 $A[1..n]$, $A[j]=j$, $1 \leq j \leq n$.

输出: $\sum_{j=1}^n A[j]$.

1. $sum \leftarrow 0$

2. **for** $j \leftarrow 1$ to n

3. $sum \leftarrow sum + A[j]$

4. **end for**

5. **return** sum

$\Theta(n)$

算法 1.15 SECOND

输入: 正整数 n .

输出: $\sum_{j=1}^n j$.

1. $sum \leftarrow 0$

2. **for** $j \leftarrow 1$ to n

3. $sum \leftarrow sum + j$

4. **end for**

5. **return** sum

$\Theta(2^k)$

$k = \lceil \log(n+1) \rceil$

